

CS3245 Information Retrieval
AY25/26, Y3S1
Notes

Sim Ray En Ryan

December 14, 2025

Contents

1	Course Organization (Lecture 0)	6
1.1	Importance of Information Retrieval (IR)	6
1.2	Evolution of Information Retrieval	6
1.3	Course Structure: Topics Covered	6
1.4	Course Details	7
2	Language Models (Lecture 1)	8
2.1	Modeling Language	8
2.2	Applications of Language Models (LMs)	8
2.3	The Unigram Model	8
2.4	Ngram Language Models (Ngram LM)	9
2.5	Markov Assumption	9
2.6	Complexity and Zero Probabilities	9
2.6.1	Complexity of Ngram LMs	9
2.6.2	Problem with Zero Probabilities	10
3	Boolean Retrieval (Lecture 2)	10
3.1	Information Retrieval (IR) Definition	10
3.2	The Challenge of Unstructured Data	10
3.3	Term-Document Incidence Matrices and Boolean Retrieval	10
3.3.1	Term-Document Incidence	10
3.3.2	Incidence Vectors	11
3.4	Information Retrieval Basics	11
3.5	The Classic Search Model	11
3.6	Evaluation Measures (Basic)	11
3.7	Scaling Up	12
3.7.1	The Problem with Incidence Matrices at Scale	12
3.7.2	Inverted Index	12
3.7.3	Inverted Index Construction Process	12
3.8	Query Processing: Merging Postings Lists	13
3.9	Query Optimization	13
4	Postings Lists and Choosing Terms (Lecture 3)	14
4.1	Faster Postings Merges: Skip Pointers/Skip Lists	14
4.1.1	Basic Merge Drawback	14
4.1.2	Skip Pointers (Skip Lists)	14
4.2	Phrase Queries and Positional Indices	14
4.2.1	Phrase Queries	14
4.2.2	Solution 1: Biword Indexes (First Attempt)	14
4.2.3	Solution 2: Positional Indexes (Standard Solution)	15
4.3	Choosing Terms for the Dictionary: Pre-processing	15

4.3.1	Text Extraction	15
4.3.2	Granularity of Indexing	15
4.3.3	Tokenization	16
4.3.4	Stop Word Removal	16
4.3.5	Normalization	16
4.3.6	Lemmatization and Stemming	16
5	Dictionaries and Tolerant Retrieval (Lecture 4)	17
5.1	Dictionary Data Structures	17
5.1.1	Naïve Dictionary (Array of Structs)	17
5.1.2	Hash Table	17
5.1.3	Tree (e.g., B-tree)	17
5.2	Tolerant Retrieval: Wildcard Queries	18
5.2.1	Simple Wildcards	18
5.2.2	General Wildcard Queries (X*Y): Permuterm Index	18
5.2.3	General Wildcard Queries: Bigram (k-gram) Index	18
5.3	Tolerant Retrieval: Spelling Correction	19
5.3.1	Uses and Flavors	19
5.3.2	Isolated Word Correction	19
5.3.3	Ngram Overlap (Jaccard Coefficient)	19
5.3.4	Context-Sensitive Correction	20
5.4	Tolerant Retrieval: Soundex	20
6	Index Construction (Lecture 5)	20
6.1	Hardware Basics	20
6.2	Scaling Index Construction	21
6.3	Blocked Sort-Based Indexing (BSBI)	21
6.4	Single-Pass In-Memory Indexing (SPIMI)	21
6.5	Distributed Indexing (MapReduce)	22
6.6	Dynamic Indexing	22
7	Index Compression (Lecture 6)	23
7.1	Collection Statistics	23
7.1.1	Heaps' Law (Vocabulary Size)	23
7.1.2	Zipf's Law (Term Frequency)	23
7.2	Why Compression?	23
7.3	Dictionary Compression	24
7.3.1	Fixed-Width vs. Dictionary-as-a-String	24
7.3.2	Blocking (for Dictionary-as-a-String)	24
7.3.3	Front Coding	24
7.4	Postings File Compression	24
7.4.1	Gap Encoding	25
7.4.2	Variable Byte (VB) Encoding	25

8	Scoring, Term Weighting and the Vector Space Model (Lecture 7)	25
8.1	Ranked Retrieval Models	25
8.1.1	Boolean Retrieval Limitations	25
8.1.2	Ranked Retrieval	26
8.2	Scoring: From Jaccard to tf-idf	26
8.2.1	Jaccard Coefficient (Take 1)	26
8.2.2	Term Frequency (tf)	26
8.2.3	Inverse Document Frequency (idf)	26
8.2.4	tf-idf Weighting	26
8.3	The Vector Space Model (VSM)	27
8.3.1	Documents and Queries as Vectors	27
8.3.2	Proximity Measure	27
8.3.3	Cosine Similarity	27
8.3.4	SMART Notation	27
9	Scoring, Term Weighting and the Vector Space Model (Lecture 8)	28
9.1	Efficient Cosine Ranking	28
9.1.1	Bottlenecks	28
9.1.2	Efficient Cosine Computation	28
9.1.3	Efficient Selection of Top K	28
9.2	Speeding Up Ranking (Approximate Methods)	28
9.2.1	Heuristic 1: Index Elimination	29
9.2.2	Heuristic 2: Champion Lists (Fancy Lists)	29
9.2.3	Tiered Indexes	29
9.2.4	Heuristic 3: Impact-Ordered Postings	29
9.2.5	Heuristic 4: Cluster Pruning	29
9.3	Incorporating Additional Ranking Information	30
9.3.1	Static Quality Scores (Authority)	30
9.3.2	Parametric and Zone Indexes	30
9.3.3	Query Term Proximity	30
10	IR Evaluation (Lecture 9)	31
10.1	Measures for a Search Engine	31
10.2	Measuring User Happiness (Relevance)	31
10.3	Unranked Retrieval Evaluation	31
10.3.1	Precision and Recall	31
10.3.2	F Measure (F_1)	31
10.4	Evaluating Ranked Results	32
10.4.1	Precision-Recall Curve	32
10.4.2	Single Number Summaries	32
10.5	Creating Test Collections	32
10.5.1	Inter-Judge (Dis)agreement	32
10.5.2	Critique of Pure Relevance	33

10.6	Evaluation at Large Search Engines	33
11	Relevance Feedback and Query Expansion, and XML IR (Lecture 10)	33
11.1	Relevance Feedback (RF) - Document Level	33
11.1.1	Explicit Relevance Feedback	33
11.1.2	Rocchio Algorithm (1971)	34
11.1.3	When Does RF Work?	34
11.1.4	Relevance Feedback Problems	34
11.1.5	Pseudo Relevance Feedback (PRF) / Blind Feedback	34
11.2	Query Expansion (QE) - Term Level	35
11.3	XML Retrieval	35
11.3.1	Structured Retrieval	35
11.3.2	Challenges in XML IR	35
11.3.3	Vector Space Model for XML IR	36
11.3.4	XML IR Evaluation (INEX)	36
12	Probabilistic IR and Language Models for IR (Lecture 11)	37
12.1	Probabilistic Approach to Retrieval	37
12.2	Probability Ranking Principle (PRP)	37
12.3	Binary Independence Model (BIM)	37
12.4	Okapi BM25 (BestMatch25)	38
12.5	Language Models (LMs) for IR	38
12.5.1	Smoothing (Mixture Model)	39
13	Crawling and Link Analysis (Lecture 12)	39
13.1	Crawling	39
13.2	Link Analysis: Anchor Text	40
13.3	Link Analysis: PageRank	40
14	Search Advertising and Duplicate Detection (Lecture 13)	41
14.1	Search Advertising	41
14.2	Duplicate Detection	41
14.2.1	Shingling and Jaccard Coefficient	42
14.2.2	Min-Hashing (for Fast Jaccard Approximation)	42

1 Course Organization (Lecture 0)

1.1 Importance of Information Retrieval (IR)

- IR is a keystone of today's tech (e.g., Google Search, Amazon, Slack, LumiNUS).
- We are in the era of big data, requiring technologies for making integration, processing, and management of massive data volumes possible.
- IR is involved in jobs everywhere, such as Data Scientist and Research Scientist, often requiring expertise in Natural Language Processing (NLP) and Machine Learning (ML).

1.2 Evolution of Information Retrieval

- **V 1.0: Directories**
 - Organized by a human.
 - Broad subject categories organized by hierarchy.
 - User selects and reads.
- **V 2.0: Search Engines**
 - Information retrieval (poor man's NLP) over billions of websites and pages.
 - Algorithms personalize the searching experience and rank results based on search criteria.
 - Uses free text search.
- **V 3.0: Conversational AI**
 - Advanced natural language understanding using machine learning and computational linguistics.
 - Knows user history and conversation state based on previous interactions.
 - Performs knowledge summarization and task completion.
 - Uses natural language conversations.

1.3 Course Structure: Topics Covered

- Boolean Retrieval
- Dictionaries and Postings Lists and Choosing Terms
- Tolerant Retrieval and Index Compression
- Index Construction and Evaluation

- Scoring and Vector Space Model
- Probabilistic IR and Language Models for IR
- Relevance Feedback and Query Expansion
- Crawling, Link Analysis, Search Advertising, Duplicate Detection, and XML IR
- A Complete Search System

1.4 Course Details

- **Lecturer:** ZHAO Jin (zhaojin@comp.nus.edu.sg)
- **Course Websites:**
 - <https://www.comp.nus.edu.sg/~cs3245> (General information, Course materials, Homework assignments)
 - <https://luminus.nus.edu.sg/> (Announcements, Internal resources, Discussion forum, Homework submissions, Webcast videos)
- **Course Activities and Grading:**
 - 13 lectures (webcast provided)
 - 6 physical tutorials (odd weeks) + 2 video tutorials
 - 4 homework assignments (individual or group)
 - 1 midterm (20%, open book)
 - 1 final exam (35%, open book)
 - Participation (5%, Tutorial + Forum)
 - Homework Assignments (40%: 5%/10%/10%/15%)
- **Collaboration Rules:**
 - Collaboration is acceptable and encouraged, but collaborators must be named on the assignment.
 - **Facebook rule:** Discuss assignments freely, even with writing on a board, but ****cannot take any written/electronic record away**** from the meeting. Must do something else for at least a half-hour before working on the assignment to ensure independent reconstruction of learned material.
- **Required Tools:**
 - **Python** programming language for all homework assignments. Python is easy to learn and useful for string manipulation (key for IR).

- **Natural Language Tool Kit (NLTK)** for some parts of the course (coded in Python).
- Students must install **Python 3** and **NLTK** on their personal computers.
- Assignments will be run on the **SoC Compute Cluster (Tembusu)**.

2 Language Models (Lecture 1)

2.1 Modeling Language

- **Traditional A.I.:** Language modeled with formal syntax and semantics (Grammar and Lexicon). Uses specific rules to define a language and helps interpret meaning (semantics) of a sentence.
- **Language Model (LM)** (in Information Retrieval and some A.I.):
 - A probabilistic model of language that is simple, often without using a formal grammar.
 - Created based on a collection of text.
 - Used to assign a probability to a sequence of words.
- **Example:** The sequence "Forsooth, there is no one I trust more" would have a high probability in a `Shakespeare_LangModel` and a low probability in a `WeChat_LangModel`.

2.2 Applications of Language Models (LMs)

- Deciding between alternatives (e.g., "Recognize speech" vs. "Wreck a nice beach").
- Speech Recognition
- Spelling Correction
- Plagiarism detection
- Predicting what products you'll browse next
- Typeahead prediction on mobile devices

2.3 The Unigram Model

- Views language as an **unordered collection of tokens** (also known as a "**bag of words**").
- Each of the n tokens contributes one count (or $1/n$) to the model.
- Outputs a count (or probability) of an input based on its individual tokens.

- $Count(input) = \sum_n Count(n)$
- $P(input) = \prod_n P(n)$
- **Limitation:** Unigrams LM do not model word order. E.g., "close my eyes" is as likely as "eyes close my".

2.4 Ngram Language Models (Ngram LM)

- Remembers **sequences of n tokens**.
- Unigram is the special case where $n = 1$.
- **Bigrams** have $n = 2$, **trigrams** have $n = 3$.
- Can be defined as a model that predicts a current word from the $n - 1$ previous context words.
- **Example:** For $P(??|$ "Please turn off your hand")
 - Unigram ($n = 1$): $P(??)$
 - Bigram ($n = 2$): $P(??|$ "hand")
 - Trigram ($n = 3$): $P(??|$ "your hand")

2.5 Markov Assumption

- The future behavior of a dynamical system only depends on its **recent history**.
- In a k^{th} -order Markov model, the next state only depends on the k most recent states.
- An N -gram model is a $(N - 1)$ -**order Markov model**.

2.6 Complexity and Zero Probabilities

2.6.1 Complexity of Ngram LMs

- Let $|V|$ be the size of the vocabulary (e.g., $|V| = 30,000$).
- Unigram LM needs to store counts/probabilities for $|V|$ words.
- Bigram LM needs to store counts/probabilities for $|V| \times |V|$ ordered length 2 phrases.
- Trigram LM needs to store counts/probabilities for $|V|^3$ ordered length 3 phrases.
- Longer Ngram models are **more accurate** but **exponentially more costly** to construct.

2.6.2 Problem with Zero Probabilities

- If a word is not seen in the training data, its probability in the LM is 0, which then causes the total probability of any sequence containing that word to be 0: $P(\text{sequence}) = \prod P(n) = 0$.
- **Add-1 smoothing (Laplace smoothing):**
 - A basic technique (not typically used in practice due to poor performance, but conceptually simple).
 - Idea: add 1 count to all entries in the LM, including those that are not seen.
 - This ensures no term has a zero probability.

3 Boolean Retrieval (Lecture 2)

3.1 Information Retrieval (IR) Definition

- IR is finding material (usually documents) of an **unstructured nature** (usually text) that satisfies an information need from within large collections (usually stored on computers).

3.2 The Challenge of Unstructured Data

- **Example Query (1680):** Which plays of Shakespeare contain the words Brutus AND Caesar but NOT Calpurnia?
- **A non-index solution (e.g., grep/text search):**
 - Slow (for large corpora).
 - Non-trivial to implement the NOT operation.
 - Other operations (e.g., finding proximity like Romans NEAR countrymen) are not feasible.
 - Ranked retrieval (returning the best documents) is not possible.

3.3 Term-Document Incidence Matrices and Boolean Retrieval

3.3.1 Term-Document Incidence

- A matrix where each row is a term and each column is a document (e.g., a Shakespeare play).
- An entry is **1** if the document contains the term, **0** otherwise.

- For the query Brutus AND Caesar but NOT Calpurnia, we can use **incidence vectors**.

3.3.2 Incidence Vectors

- Each term is represented by a vector of 1s and 0s (one dimension per document).
- To answer a query, perform a **bitwise AND** operation on the vectors.
- For Brutus AND Caesar AND NOT Calpurnia, we take the vector for Brutus, Caesar, and the **complemented** vector for Calpurnia (0s become 1s and 1s become 0s), and then AND them together.
- **Example:** 110100(Brutus) AND 110111(Caesar) AND 101111(NOT Calpurnia complement) = 100100 (The original Calpurnia vector is 010000, so the complement is 101111).

3.4 Information Retrieval Basics

- **Collection:** A fixed set of documents.
- **Goal:** Retrieve documents that are **relevant** to the user's information need and help the user complete a task.

3.5 The Classic Search Model

- The process often involves a gap between the user's ultimate **TASK** and the final **Query** sent to the Search Engine.
- A **TASK** leads to an **Info Need**, which is expressed in a **Verbal form**, and then formatted as a **Query**.
- Potential issues can arise at each step: **Misconception** (Task → Info Need), **Mis-translation** (Info Need → Verbal form), **Misformulation** (Verbal form → Query).
- Query is sent to the **Search Engine**, producing **Results** from a **Corpus**.
- If unsuccessful, the process involves **Query Refinement** (looping back to Query).

3.6 Evaluation Measures (Basic)

- **Precision (P):** Fraction of retrieved documents that are relevant to the user's information need.
- **Recall (R):** Fraction of relevant documents in the collection that are retrieved.

3.7 Scaling Up

3.7.1 The Problem with Incidence Matrices at Scale

- Consider 1 million documents, each with 1000 words. Vocabulary size $M = 500,000$.
- The term-document matrix would be $500K \times 1M$, containing half a trillion 0s and 1s (**B-I-G**).
- The matrix is **extremely sparse** (has roughly one billion 1s in total).
- Since the matrix is so sparse, only recording the positions of the 1s is a much better representation.

3.7.2 Inverted Index

- The standard solution for representing sparse data in IR.
- For each term t , store a list of all documents that contain t .
- Each document is identified by a unique **docID**.
- The lists are called **postings lists** (or simply **postings**), and an entry in a list is a **posting**.
- The main structure storing the terms and pointers to the postings lists is the **Dictionary** (or **Vocabulary** or **Lexicon**).
- Postings lists must use **variable-size storage** (e.g., linked lists, variable length arrays).
- The postings lists must be **sorted by docID** for efficient merging.

3.7.3 Inverted Index Construction Process

1. **Documents to be indexed** are fed into a **Tokenizer**.
2. **Tokenizer** outputs a **Token Stream** (sequences of characters grouped as semantic units).
3. **Linguistic modules** perform preprocessing (e.g., case folding, stemming/lemmatization) on the stream to produce **Modified tokens** (terms).
4. The modified tokens and their document IDs form a sequence of (**Term, Document ID**) pairs.
5. **Indexer steps:**
 - **Sort:** The sequence is sorted first by the **Term** and then by **docID**. This is the core indexing step.

- **Dictionary & Postings:** Multiple term entries in a single document are merged, with document frequency information (the number of documents containing the term) also stored. The structure is split into the **Dictionary** and **Postings**.

6. The result is the **Inverted Index**.

3.8 Query Processing: Merging Postings Lists

- **Boolean Query: AND** (e.g., Brutus AND Caesar)
 1. Locate Brutus in the Dictionary; retrieve its postings.
 2. Locate Caesar in the Dictionary; retrieve its postings.
 3. "Merge" the two sorted postings lists.
- **The Merge Algorithm (Intersection):**
 - Walk through the two sorted postings lists simultaneously.
 - If $docID(p_1) = docID(p_2)$, add to answer, and advance both pointers.
 - If $docID(p_1) < docID(p_2)$, advance p_1 .
 - If $docID(p_1) > docID(p_2)$, advance p_2 .
 - If list lengths are x and y , the merge takes $O(x + y)$ operations (linear time in the total number of postings entries).
 - This is only possible if the postings lists are **sorted by docID**.
- **Boolean queries** use AND, OR, and NOT to join query terms.
- **Example of industrial use:** WestLaw (legal search service) uses complex Boolean queries with proximity operators (/3 = within 3 words, /S = in same sentence).

3.9 Query Optimization

- For a query that is an AND of n terms, the best order for processing is to:
 1. Process the term with the **smallest postings list** (lowest **document frequency**).
 2. Then merge the result with the next smallest list, and continue.
- This is an effective heuristic because subsequent merges immediately cut down the set of candidate documents with the smaller set.

4 Postings Lists and Choosing Terms (Lecture 3)

4.1 Faster Postings Merges: Skip Pointers/Skip Lists

4.1.1 Basic Merge Drawback

- For the query t_1 AND t_2 , if t_1 is rare and t_2 is common, many comparisons are wasted as the pointer for t_1 walks past many entries in t_2 's list that are smaller than the current $docID(t_1)$.

4.1.2 Skip Pointers (Skip Lists)

- **Concept:** Add pointers that jump over blocks of postings entries that are guaranteed not to match the current position in the other list.
- **Implementation:** Done at indexing time. If $docID(p_1) < docID(p_2)$, check if the skip successor of p_1 is $> docID(p_2)$. If so, jump p_1 to its skip successor.
- **Tradeoff (Skip Placement):**
 - **More skips:** Shorter skip spans $\rightarrow$ more likely to skip, but costs more comparisons to check skip pointers.
 - **Fewer skips:** Fewer comparisons, but longer skip spans $\rightarrow$ fewer successful skips.
- **Heuristic:** For a postings list of length L , use $\sqrt{L}$ evenly-spaced skip pointers.

4.2 Phrase Queries and Positional Indices

4.2.1 Phrase Queries

- Need to answer queries for specific phrases (e.g., "stanford university").
- Boolean retrieval using only $\langle \text{term: docs} \rangle$ entries is insufficient.

4.2.2 Solution 1: Biword Indexes (First Attempt)

- **Concept:** Index every consecutive pair of terms (biword/bigram model using words) in the text as a phrase.
- **Example:** "I went to Stanford University" $\rightarrow$ {I went, went to, to Stanford, Stanford University}. Each biword becomes a dictionary term.
- **Query Processing:** Two-word phrase queries are immediate. Longer phrases (e.g., "stanford university palo alto") are processed as a Boolean query on biwords (stanford university AND university palo AND palo alto).
- **Issues:**

- **False positives:** A Boolean query on biwords can match documents that contain the words but not as a contiguous phrase (e.g., "stanford is a university").
- **Index blowup:** The dictionary size increases significantly.
- **Infeasible** for indexing phrases longer than biwords.

4.2.3 Solution 2: Positional Indexes (Standard Solution)

- **Concept:** In the postings, store the **position(s)** in which tokens of a term appear within the document.
- **Format:** $\langle \text{term, number of docs containing term; doc1 : position1, position2} \dots ; \text{doc2 : position1, position2} \dots ; \text{etc.} \rangle$
- **Phrase Query Processing:**
 - Extract positional postings lists for each term.
 - Use a merge algorithm recursively at the document level.
 - For a phrase query $t_1 t_2 \dots t_k$, a document d matches if for each term t_i , there is an occurrence at position p_i such that $p_2 = p_1 + 1, p_3 = p_1 + 2, \dots, p_k = p_1 + (k - 1)$.
 - This supports **proximity queries** (e.g., LIMIT! /3 STATUTE /3 FEDERAL /2 TORT where /k means "within k words of").
- **Size:** A positional index is 2–4x larger than a non-positional index, typically ~35–50% of the original text volume. It is now standard despite the size because of the power of phrase/proximity queries.

4.3 Choosing Terms for the Dictionary: Pre-processing

4.3.1 Text Extraction

- Identifying file formats (PDF/Word/HTML), language, and character sets. Usually handled by vendor libraries.

4.3.2 Granularity of Indexing

- What is the unit document? A file, email, paragraph, etc.?
- Too coarse-grained $\rightarrow$ everything matches, high precision needed within hits.
- Too fine-grained $\rightarrow$ nothing matches.
- Must be decided based on the projected use of the IR engine.

4.3.3 Tokenization

- **Input:** Raw text (e.g., "Friends, Romans, Countrymen, lend me your ears;")
- **Output:** Tokens (e.g., Friends, Romans, Countrymen, lend, me, your, ears)
- A **token** is an instance of a sequence of characters grouped as a useful semantic unit, and a candidate for an index entry.
- **Issues:** Handling apostrophes (Finland's), hyphens (Hewlett-Packard), spaces (San Francisco), numbers/dates (3/20/13), and non-English language complexities (German compounds, Chinese/Japanese segmentation, Arabic/Hebrew bidirectionality/ligatures).

4.3.4 Stop Word Removal

- Excluding the most common words (e.g., **the**, **a**, **and**, **to**, **be**) from the dictionary as they have little semantic content.
- **Trend:** Moving away from stop word removal because modern compression/query optimization techniques minimize the cost.
- **Needed for:** Phrase queries ("King of Denmark"), song titles ("Let it be"), and relational queries ("flights to London").

4.3.5 Normalization

- Converting tokens into a standard **term** (the entry in the dictionary).
- **Equivalence Classes:** Implicitly defined by deleting certain characters (e.g., U.S.A., USA $\rightarrow$ USA).
- **Removing accents/diacritics:** Often best to normalize (de-accent) to how users most often type their queries (e.g., Tuebingen, Tübingen $\rightarrow$ Tubingen).
- **Case-folding:** Reducing all letters to lower case. Often best to lowercase everything as user queries are typically lowercase.
- **Alternative:** Asymmetric expansion (e.g., search for "Windows" only searches for "Windows", but search for "windows" searches for "Windows, windows, window").

4.3.6 Lemmatization and Stemming

- **Lemmatization:** Reducing inflectional/variant forms to a base/dictionary form (e.g., am, are, is $\rightarrow$ be). Implies "proper" reduction.
- **Stemming:** Reducing terms to their "roots" before indexing using crude affix chopping (e.g., automate(s), automatic, automation $\rightarrow$ automat). Language dependent.

- **Porter's Algorithm:** Most common for English. Consists of sequential phases of reductions; checks the length of the resulting word.
- **Effectiveness:** Mixed results for English (helps recall, harms precision), but highly useful for highly inflected languages like Finnish (up to 30% gain).

5 Dictionaries and Tolerant Retrieval (Lecture 4)

5.1 Dictionary Data Structures

- The dictionary stores the term vocabulary, document frequency, and pointers to each postings list.

5.1.1 Naïve Dictionary (Array of Structs)

- **Structure:** `char[20] (Term) int (Doc Freq) Postings Pointer`
- **Drawbacks:**
 - Fixed-size term length (wasteful space, can't handle long words).
 - **Slow to access** - requires linear scan at query time.

5.1.2 Hash Table

- **Pros:** Lookup is fast, $O(1)$.
- **Cons:**
 - Not tolerant: No easy way to find minor variants (e.g., judgment/judgement).
 - No prefix search (e.g., finding terms starting with `mon`).
 - Requires expensive rehashing if vocabulary grows too large.
- **Result:** Not very tolerant (cannot handle wildcards easily).

5.1.3 Tree (e.g., B-tree)

- Requires lexicographical ordering of terms.
- **Pros:**
 - Solves the prefix problem (e.g., finding all terms starting with `hyp`).
 - More tolerant (easier for approximate matching).
- **Cons:** Slower lookup, $O(\log M)$ (requires a balanced tree).

- **B-tree:** Specialized tree structure that mitigates the cost of rebalancing inherent in binary trees.

5.2 Tolerant Retrieval: Wildcard Queries

5.2.1 Simple Wildcards

- **mon*:** Find all words beginning **mon**. Easy with a B-tree lexicon: retrieve all words in range $\text{mon} \leq w < \text{moo}$.
- ***mon:** Find words ending in **mon**. Needs an **additional B-tree** built on **reversed terms**. Retrieve range $\text{nom} \leq w < \text{non}$.
- **pro*cent:** Use forward B-tree for **pro*** AND reverse B-tree for ***cent**, then intersect the resulting term sets.

5.2.2 General Wildcard Queries (X*Y): Permuterm Index

- **Goal:** Transform the query so the wildcard ***** always occurs at the end.
- **Mechanism:** For every term, generate a set of rotations by placing a special symbol **\$** at the end and then rotating. Index all rotations.
- **Example:** Term **hello** is indexed under: **hello\$, ello\$h, llo\$he, lo\$hel, o\$hell, \$hello**.
- **Query hel*o:** X=**hel**, Y=**o**. Rotate to $Y\$X^* = \text{o\$hel*}$. This is looked up in the B-tree as a prefix search.
- **Problem:** Lexicon size **blows up**, proportional to the average word length.

5.2.3 General Wildcard Queries: Bigram (k-gram) Index

- **Mechanism:**
 - Enumerate all k-grams (sequences of k chars) occurring in any term (using **\$** as a boundary marker).
 - Maintain a second inverted index from bigrams to the dictionary terms that contain them.
- **Query Processing (mon*):** Run as a conjunction of bigrams: **\$m** AND **mo** AND **on**.
- **Result:** Retrieves all matching terms (e.g., **among**, **amortize**, **madden**) but may include **false positives** (e.g., **moon**).
- **Post-filtering:** All surviving enumerated terms must be verified against the original wildcard pattern.
- **Pros:** Fast, space efficient (compared to permuterm).

5.3 Tolerant Retrieval: Spelling Correction

5.3.1 Uses and Flavors

- **Uses:** Correcting documents being indexed (especially OCR'ed), and correcting user queries.
- **Flavors:**
 - **Isolated word:** Checks each word on its own. Fails to catch typos resulting in correctly spelled words (e.g., from → form).
 - **Context-sensitive:** Looks at surrounding words (e.g., I flew **form** Heathrow to Narita).

5.3.2 Isolated Word Correction

- **Premise:** There is a lexicon (e.g., standard dictionary or indexed corpus) of correct spellings.
- **Goal:** Given sequence Q, return words in the lexicon closest to Q.
- **Closest Defined by:**
 1. **Edit Distance (Levenshtein Distance):** Minimum number of operations (Insert, Delete, Replace, Transposition) to convert one string to another. Found by dynamic programming.
 2. **Weighted Edit Distance:** Weights the operations based on the characters involved (e.g., keyboard proximity, OCR errors).
 3. **Ngram Overlap (Jaccard Coefficient):**

5.3.3 Ngram Overlap (Jaccard Coefficient)

- **Mechanism:**
 1. Enumerate all ngrams in the query string and lexicon terms.
 2. Use the ngram index to retrieve all lexicon terms matching any of the query ngrams.
 3. Compute the **Jaccard Coefficient** for overlap.
- **Jaccard Coefficient** $J(X, Y) = \frac{|X \cap Y|}{|X \cup Y|}$. Ranges from 0 (disjoint) to 1 (identical).
- The final term selection depends on a threshold (e.g., *Jaccard* > 0.8).

5.3.4 Context-Sensitive Correction

- **Method:**

1. Retrieve dictionary terms close (by weighted edit distance) to each query term.
2. Enumerate all possible resulting phrases with one word "corrected" at a time.

- **Phrase-based approach:** Break phrase queries into conjunctions of biwords, then look for biwords that need only one term corrected.
- **Presentation:** Often uses a "Did you mean ...?" suggestion. Corrections can be ranked by the **hit count** (the correction with the most document hits).

5.4 Tolerant Retrieval: Soundex

- **Purpose:** A class of heuristics to expand a query into **phonetic equivalents** (mainly for names).
- **Algorithm (English):** Converts a word into a 4-character code (First Letter + 3 Digits).
- **Process (Simplified):**
 1. Retain the first letter.
 2. Map certain consonants to digits (1-6). Vowels/H/W/Y $\rightarrow$ 0.
 3. Repeatedly remove one out of each pair of consecutive identical digits.
 4. Remove all zeros.
 5. Pad with trailing zeros to get $\langle \text{Letter} \rangle \langle \text{Digit} \rangle \langle \text{Digit} \rangle \langle \text{Digit} \rangle$.
- **Usefulness:** Not very good for general IR or spelling correction, but okay for high-recall tasks where phonetic matching is critical.

6 Index Construction (Lecture 5)

6.1 Hardware Basics

- **Bottleneck:** Hard Drive Storage.
 - **Seek Time:** Time to move to a random location ($\sim 8\text{ms}$ HDD, $\sim 0.1\text{ms}$ SSD).
 - **Transfer Time:** Time to transfer a data block ($\sim 6\ \mu\text{s}/\text{byte}$ HDD, $\sim 2\ \mu\text{s}/\text{byte}$ SSD).
- **Implications for IR:**
 - Access to data in memory is much faster than disk.

- Disk seeks are expensive $\rightarrow$ transferring one large chunk is faster than many small chunks.
- Disk I/O is block-based (typically 4KB).
- For large-scale IR, commodity machines are preferred (cheaper) over fault-tolerant machines (fault tolerance is achieved via distributed architecture).

6.2 Scaling Index Construction

- **Problem:** In-memory index construction does not scale to very large collections (e.g., New York Times over 150 years of newswire).
- **Memory Requirement (RCV1):** $T = 100$ million (`term`, `docID`) pairs. At ~ 11.5 bytes/pair, this is ~ 1.1 GB, which can fit in memory.
- For larger collections, intermediate results must be stored on disk.
- **Constraint:** Simple disk sorting of 100M records is too slow due to too many random disk seeks $\rightarrow$ need an **external sorting algorithm**.

6.3 Blocked Sort-Based Indexing (BSBI)

- **Goal:** Minimize disk seeks during sorting.
- **Steps:**
 1. Map variable length terms to fixed-length **termIDs** (e.g., 4 bytes).
 2. Represent postings entries as fixed-length (`termID`, `docID`) records (e.g., 8 bytes).
 3. **Blocking:** Accumulate records for a block size (e.g., 10M records) until memory is full.
 4. **In-memory Sort:** Sort each block by `termID` (then `docID`) using Quicksort.
 5. **Write Runs:** Convert sorted blocks into compressed postings lists and write to disk as a sorted "run".
 6. **Merge Runs:** Merge all sorted runs (typically using an n -way merge from disk) into one large inverted index.

6.4 Single-Pass In-Memory Indexing (SPIMI)

- **Motivation for SPIMI:** BSBI assumes the dictionary (for `term` $\rightarrow$ `termID` mapping) can be kept in memory. For very large collections, this assumption might fail.
- **Key Ideas:**

1. **No Global Dictionary:** Generate separate dictionaries for each block.
 2. **In-memory Postings Lists:** Build the dictionary and postings list simultaneously in memory for each block.
 3. **One Pass:** Process the input documents only once.
 4. **Write Index:** Write out a complete, compressed inverted index for each block to disk.
- **Final Step:** Merge the mini-indices from each block into one large index (analogous to BSBI's merge step).
 - SPIMI benefits greatly from **compression** of both terms and postings.

6.5 Distributed Indexing (MapReduce)

- **Application:** Web-scale indexing where collections are massive and individual machines are fault-prone.
- **Architecture:** Master machine directs indexing job to a pool of worker machines.
- **Parallel Tasks (MapReduce Framework):**
 1. **Parsers (Map Phase):**
 - Master assigns a document split (subset of docs) to an idle parser.
 - Parser reads docs and emits (**term**, **docID**) pairs.
 - Pairs are partitioned (e.g., by first letter range **a-f**) and written to disk.
 2. **Inverters (Reduce Phase):**
 - An inverter collects all (**term**, **docID**) pairs for one term-partition.
 - Sorts the postings and writes them to the final postings lists on disk.

6.6 Dynamic Indexing

- **Problem:** Collections are rarely static (new/modified/deleted documents). Re-indexing from scratch is impractical.
- **Simple Approach:**
 - Maintain a **"big" main index**.
 - New docs go into a **"small" auxiliary index** (in memory).
 - Search across both, merging results.
 - **Deletions:** Handled by an **invalidation bit-vector** that filters search results.

- Periodically, merge the auxiliary index into the main index.
- **Logarithmic Merge:**
 - Idea: Maintain a series of indexes on disk ($I_0, I_1, \dots$) where each is twice as large as the previous one. New batches are merged into the appropriate sized index recursively.
 - **Cost:** Each posting is merged $O(\log T)$ times $\rightarrow$ index construction time is $O(T \log T)$.
 - **Tradeoff:** Query processing now requires merging $O(\log T)$ indexes instead of $O(1)$ for the simple approach.
- **Further Issue:** Collection-wide statistics (e.g., document frequency df_t for idf/ranking) are hard to maintain across multiple indexes.

7 Index Compression (Lecture 6)

7.1 Collection Statistics

7.1.1 Heaps' Law (Vocabulary Size)

- **Relationship:** $M = kT^b$
- M is the size of the vocabulary (number of distinct terms).
- T is the number of tokens (total word occurrences) in the collection.
- k and b are empirical constants (typically $30 \leq k \leq 100$ and $b \approx 0.5$).
- In a log-log plot, it predicts a linear relationship with slope $\approx 1/2$.

7.1.2 Zipf's Law (Term Frequency)

- **Relationship:** The i^{th} most frequent term has frequency proportional to $1/i$.
- $cf_i \propto 1/i = K/i$
- cf_i is the **collection frequency** (number of occurrences in the whole collection).
- Implies that a few terms are very frequent, and many terms are very rare.
- Leads to a linear relationship between $\log cf_i$ and $\log i$.
-]

7.2 Why Compression?

- Use less disk space (saves money).

- Keep more data in memory (increases speed).
- Increase speed of data transfer from disk: [read compressed data|decompress] is faster than [read uncompressed data].

7.3 Dictionary Compression

7.3.1 Fixed-Width vs. Dictionary-as-a-String

- **Fixed-Width Drawback:** Wastes substantial space on the term column (e.g., using 20 bytes/term when the average is 8 chars).
- **Dictionary-as-a-String:**
 - Stores the dictionary as one long string of concatenated terms.
 - Dictionary entries now only need pointers to the term's location in the string.
 - Saves significant dictionary space ($\sim 60\%$).

7.3.2 Blocking (for Dictionary-as-a-String)

- **Concept:** Store pointers only to every k^{th} term string (the start of a block).
- **Space Tradeoff:** Saves on pointers for $k - 1$ terms, but adds overhead for storing term lengths.
- **Query Time Tradeoff:** Search now requires binary search to the block boundary, followed by a linear scan through the terms in the block. Larger k saves more space but slows down query time due to longer linear scan.

7.3.3 Front Coding

- **Concept:** Exploits the long common prefix of sorted words by storing only the differences.
- Used for the last $k - 1$ terms within a block of size k .
- **Example:**

7.4 Postings File Compression

- **Problem:** Postings file is much larger than the dictionary (factor of at least 10).
- **Goal:** Use much less than the native bit size (e.g., 20 bits/docID).

7.4.1 Gap Encoding

- The postings list is already sorted by **docID**.
- Instead of storing the absolute **docID**, store the **gap** (difference) between consecutive **docIDs**.
- **Implication from Zipf's Law:**
 - **Common terms:** Appear in many documents → many **docIDs**, **small gaps**.
 - **Rare terms:** Appear in few documents → few **docIDs**, **large gaps**.
- This varying gap size suggests using a variable-length encoding scheme.

7.4.2 Variable Byte (VB) Encoding

- **Concept:** Encodes every integer (gap) with close to the fewest bytes needed.
- **Mechanism:** Dedicate 1 bit as a **continuation bit** c (e.g., in a byte).
 - If $G \leq 127$, encode it in the 7 available bits and set $c = 1$.
 - Else, encode G 's lower-order 7 bits and use additional bytes for higher-order bits (set $c = 0$).
 - The last byte of the integer representation has $c = 1$.
- **Key Property:** VB-encoded postings are **uniquely prefix-decodable** (can tell where one integer ends and the next begins).
- VB codes are widely used for balancing variable-length coding with computer memory alignment.

8 Scoring, Term Weighting and the Vector Space Model (Lecture 7)

8.1 Ranked Retrieval Models

8.1.1 Boolean Retrieval Limitations

- Documents either match or don't (→ precise, good for experts).
- **Feast or Famine:** Often results in too few (0) or too many (1000s) results.
- Bad for the majority of users (especially web search) who are not proficient in Boolean query language.

8.1.2 Ranked Retrieval

- The system returns an **ordering** over the documents with respect to a query.
- Documents are ranked by a **score** (e.g., in $[0, 1]$) that measures how well the document and query "match."
- Typically associated with **free text queries**.

8.2 Scoring: From Jaccard to tf-idf

8.2.1 Jaccard Coefficient (Take 1)

- Measures the overlap of the set of terms in a document D and a query Q : $J(Q, D) = \frac{|Q \cap D|}{|Q \cup D|}$.
- **Drawback:** Does **not** consider:
 - **Term Frequency (tf):** The number of times a term appears within a document.
 - **Document Frequency (df):** The number of documents containing the term.

8.2.2 Term Frequency (tf)

- $tf_{t,d}$: the raw count of term t in document d .
- Raw frequency is problematic: a doc with 10 occurrences is more relevant than 1, but **not** 10 times more relevant.
- **Log-frequency weighting scheme:** $w_{t,d} = \log_{10}(1 + tf_{t,d})$ (or 0 if $tf_{t,d} = 0$). Dampens the effect of high raw frequency.

8.2.3 Inverse Document Frequency (idf)

- **Intuition:** Rare terms are more informative than frequent terms.
- df_t : the number of documents containing t .
- **idf formula:** $idf_t = \log_{10}(N/df_t)$ (where N is the total number of documents).
- Dampens the effect of high df_t (common words like **the** have $idf \approx 0$).
- idf affects the ranking only for queries with **at least two terms**.

8.2.4 tf-idf Weighting

- **Formula:** $w_{t,d} = (1 + \log_{10} tf_{t,d}) \times \log_{10}(N/df_t)$
- It is the **best known weighting scheme in IR**.

- Increases with term rarity (*idf*).
- Increases with term frequency within a document (*tf*).

8.3 The Vector Space Model (VSM)

8.3.1 Documents and Queries as Vectors

- Each document is represented by a **real-valued vector** of tf-idf weights $\in R^{|V|}$.
- The dimensions of the space are the terms (vocabulary $|V|$).
- These are typically **very sparse vectors**.
- **Bag of Words Model:** This representation does **not** consider the ordering of words (a limitation compared to positional indexes).
- **Queries** are treated as **mini-documents** and also represented as vectors.

8.3.2 Proximity Measure

- Rank documents according to their **proximity** to the query vector.
- **Euclidean Distance** is a **bad idea** because it gives a large distance for vectors of different lengths (i.e., short vs. long documents with the same content distribution).
- **Angle (Cosine Similarity)** is used instead. A smaller angle corresponds to maximal similarity.

8.3.3 Cosine Similarity

- Documents are ranked in decreasing order of $\cos(\vec{q}, \vec{d})$.
- **Formula:** $\cos(\vec{q}, \vec{d}) = \frac{\vec{q} \cdot \vec{d}}{|\vec{q}| |\vec{d}|} = \frac{\sum_{i=1}^{|V|} q_i d_i}{\sqrt{\sum_{i=1}^{|V|} q_i^2} \sqrt{\sum_{i=1}^{|V|} d_i^2}}$
- q_i and d_i are the tf-idf weights of term i in the query and document, respectively.
- **Length Normalization:** Dividing by the vector length $|\vec{v}|$ ensures that the magnitude does not affect the similarity.
- For **length-normalized vectors**, cosine similarity is simply the **dot product**: $\cos(\vec{q}, \vec{d}) = \sum_{i=1}^{|V|} q_i d_i$.

8.3.4 SMART Notation

- Used to denote the combination of weighting schemes for documents and queries in the form `ddd.qqq`.

- **Document weight acronyms (ddd):** `l` for log-frequency, `n` for no *idf*, `c` for cosine normalization.
- **Query weight acronyms (qqq):** `l` for log-frequency, `t` for *idf*, `c` for cosine normalization.
- **Standard Scheme:** `lnc.ltc`
 - Document (`lnc`): log tf, no *idf*, cosine normalization.
 - Query (`ltc`): log tf, *idf*, cosine normalization.

9 Scoring, Term Weighting and the Vector Space Model (Lecture 8)

9.1 Efficient Cosine Ranking

9.1.1 Bottlenecks

- **Primary Computational Bottleneck:** The cosine computation, which can be expensive for all documents with non-zero scores (J).
- The goal is to find the K docs in the collection "nearest" to the query (i.e., the K largest query-doc cosines) efficiently.

9.1.2 Efficient Cosine Computation

- **Unweighted Query Optimization:** If all query terms are unweighted ($\omega_{t,q} = 1$), the score calculation simplifies significantly: $\frac{\sum_{t \in q} \omega_{t,d}}{|d|}$ (no expensive multiplication, only addition).
- **Pre-computation:** The document length $|d|$ is pre-computed and stored.

9.1.3 Efficient Selection of Top K

- The task is to select the K largest values from the set J (documents with non-zero cosine).
- **Heaps (Min-Heap):** A min-heap of size K stores the K largest scores seen so far.
- **Complexity:** $O(J + K \log J)$ operations (vs. $O(J \log J)$ for full sorting).

9.2 Speeding Up Ranking (Approximate Methods)

- To avoid computing scores for all J documents, heuristics are used to find a subset A of contenders ($K < |A| \ll N$) and then return the top K from A . This **prunes non-contenders**.

9.2.1 Heuristic 1: Index Elimination

- **A. High-idf query terms only:** Only accumulate scores from terms with high *idf* (e.g., ignoring common words like **in**, **the**). Low *idf* terms have many postings entries, so this greatly reduces the set *A*.
- **B. Docs containing many query terms:** For multi-term queries, only compute scores for documents containing at least some threshold number of query terms (e.g., 3 out of 4). This imposes a "soft conjunction."

9.2.2 Heuristic 2: Champion Lists (Fancy Lists)

- **Pre-processing:** For each term *t*, pre-compute and store the *r* documents of highest weight ($\omega_{t,d}$) in *t*'s postings list. This subset is the **champion list**.
- **Query Time:** Only compute scores for documents that appear in the champion list of **at least one** query term.

9.2.3 Tiered Indexes

- **Concept:** Generalizes champion lists by breaking the inverted index into a hierarchy of tiers of decreasing importance.
- **Query Time:** Use only the top tier unless it provides insufficient results, then drop to lower tiers.

9.2.4 Heuristic 3: Impact-Ordered Postings

- **Concept:** Sort each postings list by descending term weight ($\omega_{t,d}$), often called **impact**.
- **Problem:** Postings lists are no longer sorted by **docID**, preventing simultaneous traversal and merging.
- **A. Early Termination:** Stop traversing a postings list after a fixed number of documents *r* or when $\omega_{t,d}$ drops below a threshold. Then compute scores for the union of all resulting document sets.
- **B. IDF Ordered Terms:** Process query terms in order of decreasing *idf* (high *idf* terms contribute most to score).

9.2.5 Heuristic 4: Cluster Pruning

- **Pre-processing (Offline):**
 1. Pick *N* documents at random as **leaders**.

2. Assign every other document (a **follower**) to its nearest leader (forming clusters).

- **Query Processing (Online):**

1. Find the query's nearest leader L .
2. Seek K nearest documents only from among L 's followers (and L itself).

9.3 Incorporating Additional Ranking Information

9.3.1 Static Quality Scores (Authority)

- Documents should be ranked by relevance **and** authority.
- **Authority Score $g(d)$:** A query-independent quality score (e.g., PageRank, number of citations) assigned to each document d in $[0, 1]$.
- **Net Score:** $\text{net-score}(q, d) = g(d) + \cos(q, d)$ (or other linear combinations).
- **Optimization (Ordering):** Order all postings by $g(d)$ (then by docID). Since this is a **common ordering** for all terms, it is possible to concurrently traverse postings for intersection/scoring.

9.3.2 Parametric and Zone Indexes

- **Field (Parametric Index):** Metadata with a finite set of values (e.g., `Year = 1818`). Postings are built for each field value. Queries treat fields as conjunctions.
- **Zone (Zone Index):** A region with an arbitrary amount of text (e.g., `Title`, `Abstract`, `Reference`). Inverted indexes are built on zones, allowing queries like `merchant in the title zone`.

9.3.3 Query Term Proximity

- **Concept:** Users prefer documents where query terms occur close to each other.
- **Scoring:** Documents can be scored based on w , the smallest window in a document containing all query terms (smaller w is better).
- **Query Parsers:** A free text query (e.g., `NUS open day`) may spawn a sequence of increasingly relaxed queries:
 1. Phrase query: `"NUS open day"`
 2. Proximity queries: `"NUS open" AND "open day"`
 3. Vector space query: `NUS open day`

The matching documents are then ranked by VSM.

10 IR Evaluation (Lecture 9)

10.1 Measures for a Search Engine

- **Measurable Criteria:** Indexing speed (docs/hour), search speed (latency vs. index size), expressiveness of query language (ability to express complex needs), cost.
- **Key Measure:** User happiness (or satisfaction).

10.2 Measuring User Happiness (Relevance)

- **Proxy:** Relevance of search results.
- **Relevance Measurement Requirements (Test Collection):**
 1. A set **document collection** (e.g., TREC).
 2. A suite of **queries** (must be relevant to the documents).
 3. A binary or graded **relevance assessment** (Relevant/Non-relevant) for each query-document pair (usually done by human judges).
- **Important Note:** Relevance is assessed relative to the original **information need**, not the query itself.

10.3 Unranked Retrieval Evaluation

10.3.1 Precision and Recall

- **Precision (P):** $\frac{\# \text{ of relevant doc retrieved}}{\text{total } \# \text{ of documents retrieved}} = \frac{tp}{tp+fp}$
- **Recall (R):** $\frac{\# \text{ of relevant doc retrieved}}{\text{total } \# \text{ of relevant documents}} = \frac{tp}{tp+fn}$
- **tp** (true positive), **fp** (false positive), **fn** (false negative), **tn** (true negative).
- **Accuracy** ($\frac{tp+tn}{tp+fp+tn+fn}$) is a bad measure for IR because tn (correctly non-retrieved) is usually massive, leading to high accuracy even if performance is bad.
- In a good system, **P** generally **decreases** as **R** increases (precision/recall tradeoff).

10.3.2 F Measure (F_1)

- A combined measure that assesses the precision/recall tradeoff.
- It is the **weighted harmonic mean** of P and R .
- **Formula (Balanced F_1 where $\beta = 1$):** $F_1 = \frac{2PR}{P+R}$
- The harmonic mean is a conservative average, punishing low scores more (e.g., $P = 0.8, R = 0.2 \rightarrow F_1 = 0.32$).

10.4 Evaluating Ranked Results

10.4.1 Precision-Recall Curve

- Plotting P at different R levels.
- P is calculated every time a relevant document is retrieved.
- **Interpolated Precision:** To deal with local increases/fluctuations in precision, the curve is smoothed by taking the maximum precision to the right of the current recall value.

10.4.2 Single Number Summaries

- **Precision-at- k ($P@k$):** Precision of the top k results. Appropriate for web search where users only care about the first few pages.
- **11-point interpolated average precision:** Standard in early TREC competitions; averages interpolated precision at 11 recall levels (0.0, 0.1, ..., 1.0).
- **Mean Average Precision (MAP):**
 - The **standard evaluation measure** in research.
 - For a single query, it's the average of the precision values obtained each time a relevant document is retrieved.
 - For a collection, it's the arithmetic average over all queries (macro-averaging).
- **R-precision:** Precision of the top Rel docs returned, where Rel is the total number of relevant documents in the collection for that query.

10.5 Creating Test Collections

10.5.1 Inter-Judge (Dis)agreement

- Human judges are subjective; need a way to measure agreement.
- **Kappa Measure (K):** Measures agreement among judges, corrected for chance agreement.
- **Formula:** $K = \frac{P(A) - P(E)}{1 - P(E)}$
- $P(A)$: proportion of time judges agree.
- $P(E)$: proportion of agreement expected by chance.
- $K > 0.8 \rightarrow$ Good agreement. $0.67 < K < 0.8 \rightarrow$ Tentative conclusions.

10.5.2 Critique of Pure Relevance

- **Marginal Relevance:** A better measure of utility, as a document can be highly relevant but redundant (e.g., a near-duplicate).
- Evaluating using facts/entities is harder but measures true relevance more directly.

10.6 Evaluation at Large Search Engines

- **Measures:** Often use $P@k$ or measures that reward higher ranks (e.g., NDCG, MRR).
- **Non-relevance measures:** Clickthrough on the first result (reliable in the aggregate).
- **A/B Testing:**
 - **Purpose:** To test a single innovation (change to the system).
 - Divert a small proportion of traffic (e.g., 1%) to the new system.
 - Evaluate using an "automatic" **Overall Evaluation Criterion (OEC)** like clickthrough rate.
 - Generally the most trusted evaluation methodology for large search engines.
 - **Pitfall:** Choosing the wrong OEC can lead to invalid conclusions (e.g., a drop in OEC leads to the correct business outcome).
 - **HiPPO:** Highest Paid Person's Opinion; good practices require data and experimentation over mere opinion.

11 Relevance Feedback and Query Expansion, and XML IR (Lecture 10)

11.1 Relevance Feedback (RF) - Document Level

11.1.1 Explicit Relevance Feedback

- **Mechanism:** User provides explicit feedback by marking some retrieved documents as **Relevant** (D_r) and others as **Non-relevant** (D_{nr}).
- **Goal:** Use this feedback to mathematically generate a new, **refined query** (q_m) that is a better representation of the user's information need.
- **Key Concept: Centroid:** The center of mass for a set of documents D : $\vec{c} = \frac{1}{|D|} \sum_{\vec{d} \in D} \vec{d}$.

11.1.2 Rocchio Algorithm (1971)

- Uses the vector space model to compute the optimal query vector ($\vec{q}_{\text{opt}}$) that maximizes the separation between relevant (C_r) and non-relevant (C_{nr}) documents in the vector space.
- **Theoretically Best Query:** $\vec{q}_{\text{opt}} = \frac{1}{|C_r|} \sum_{\vec{d} \in C_r} \vec{d} - \frac{1}{|C_{nr}|} \sum_{\vec{d} \in C_{nr}} \vec{d}$.
- **Refined Query Formula:** In practice, using judgments from a few documents:

$$\vec{q}_m = \alpha \vec{q}_0 + \beta \left(\frac{1}{|D_r|} \sum_{\vec{d} \in D_r} \vec{d} \right) - \gamma \left(\frac{1}{|D_{nr}|} \sum_{\vec{d} \in D_{nr}} \vec{d} \right)$$

- $\vec{q}_0$: Original query vector. α, β, γ : tuning weights.
- Many systems ignore negative feedback ($\gamma = 0$), and negative term weights in $\vec{q}_m$ are often ignored (set to 0).

11.1.3 When Does RF Work?

- RF is often very useful for one round.
- **Assumptions:**
 1. User's initial query at least partially works.
 2. (Non-)relevant documents are cohesive (i.e., relevant documents are similar to each other).

11.1.4 Relevance Feedback Problems

- Long queries ($\vec{q}_m$) are inefficient to process, leading to long response times. (Hack: reweight only the top # of prominent terms).
- Users are reluctant to provide explicit feedback.

11.1.5 Pseudo Relevance Feedback (PRF) / Blind Feedback

- **Concept:** Automates RF by assuming the **top k retrieved documents are relevant** (D_r).
- **Process:** Retrieve initial results $\rightarrow$ Assume top k are D_r (and the rest are D_{nr}) $\rightarrow$ Apply Rocchio formula $\rightarrow$ Re-rank.
- Works very well on average but can fail dramatically for some queries (query drift).
-]

11.2 Query Expansion (QE) - Term Level

- **Concept:** Additional input (good/bad search term) is given on words or phrases to augment the original query $\vec{q}_0$.
- **Methods for Query Augmentation:**
 1. **Manual Thesaurus:** Using an editorially built thesaurus (e.g., MedLine, WordNet) to expand terms with synonyms (e.g., `feline` $\rightarrow$ `feline cat`).
 2. **Automatic Thesaurus Generation:**
 - Generates a thesaurus by analyzing co-occurrence patterns in the documents (distributional similarity).
 - Simplest method: computing term-term similarity in a co-occurrence matrix $C = AA^T$ where A is the term-document matrix.
 - **Problem:** Term ambiguity can introduce irrelevant correlated terms (e.g., `Apple computer` $\rightarrow$ `Apple red fruit computer`).
- **Effect:** Generally increases **recall**, but may decrease **precision** if terms are ambiguous.

11.3 XML Retrieval

11.3.1 Structured Retrieval

- **Premise:** Documents are **structured** (e.g., XML) while queries can be structured or unstructured.
- **XML Document Structure:** Composed of nested **elements** (internal nodes encoding structure/metadata) and **text** (leaf nodes). Elements can have **attributes**.

11.3.2 Challenges in XML IR

- **Document Parts to Retrieve:** Users often want parts of documents (XML elements), not the entire document.
- **Structured Document Retrieval Principle:** A system should always retrieve the **most specific part of a document** that answers the query.
- **Indexing Unit (4 methods):** Deciding what constitutes a retrievable/indexable "document" unit in a nested structure (e.g., Non-overlapping pseudo-documents, Top down, Bottom up, or Index all elements).
- **Nested elements:** Results are often highly redundant due to nested elements. Techniques are needed to remove nested elements in postprocessing or collapse them in the results list.

- **Term Statistics (idf):** Term statistics (especially *idf*) may need to distinguish between different contexts of a term (e.g., **Gates** under **author** vs. **section**). A compromise is often made using the parent node to distinguish contexts.

11.3.3 Vector Space Model for XML IR

- **Main Idea: Lexicalized Subtrees:** Dimensions in the vector space are defined by a **structural term** $\langle c, t \rangle$: a pair of XML-context c (path from root to node) and vocabulary term t .
- **Context Resemblance (CR):** Measures the similarity between a query path c_q and a document path c_d . $CR(c_q, c_d) = \frac{1+|c_q \cap c_d|}{1+|c_q \cup c_d|}$. It is 1.0 if paths are identical.
- **Similarity Measure (SimNoMerge):** A variant of the cosine measure computed as a weighted sum over all structural terms:

$$\text{SimNoMerge}(q, d) = \sum_{t \in V} \sum_{c \in B} CR(c_q, c_d) \times \text{weight}(q, t, c) \times \text{weight}(d, t, c)$$

This is not a true cosine (can be > 1.0) because each context is calculated separately ("No Merge").

11.3.4 XML IR Evaluation (INEX)

- **INEX:** Initiative for the Evaluation of XML retrieval (standard benchmark).
- **Topic Types:**
 1. **Content-Only (CO):** Regular keyword queries.
 2. **Content-and-Structure (CAS):** Queries with structural constraints (e.g., `scene#title#"Macbeth"`).
- **Relevance Dimensions (Orthogonal):**
 1. **Component Coverage:** Evaluates if the element is structurally correct (Exact, Too small, Too large, No coverage).
 2. **Topical Relevance:** Relevance level (Highly, Fairly, Marginally, Nonrelevant).
- The final judgment is a **digit-letter code** (e.g., 2S) which is converted into a score using a **quantization function** (Q) to allow for **partial relevance**.

12 Probabilistic IR and Language Models for IR (Lecture 11)

12.1 Probabilistic Approach to Retrieval

- **Goal:** Estimate the probability of relevance $P(R = 1|d, q)$ (i.e., how likely the document is relevant to the query).
- **Foundation:** Probability theory provides a principled way to reason under uncertainty about document relevance.
- **Assumption:** Binary notion of relevance ($R_{d,q} = 1$ if relevant, 0 otherwise).

12.2 Probability Ranking Principle (PRP)

- **Principle:** The retrieval effectiveness will be the best obtainable if documents are ranked decreasingly by their estimated probability of relevance $P(R = 1|d, q)$.

12.3 Binary Independence Model (BIM)

- **Assumptions:**
 - **Binary (Boolean):** Documents and queries are represented as binary term incidence vectors $\vec{x}$ ($x_t = 1$ if term t occurs, 0 otherwise).
 - **Conditional Independence:** The presence or absence of a word is independent of all others, **given the query and relevance category** (a Naïve Bayes assumption).
 - **Out-of-query terms:** Terms not in the query are equally likely to occur in relevant and non-relevant documents ($p_t = u_t$ if $q_t = 0$).
 - **Document relevance values** are independent.
- **Ranking Function (RSV):** Under these assumptions, documents are ranked by their **Retrieval Status Value (RSV)**, which is a sum of log odds ratios:

$$\text{RSV}_d = \sum_{t: x_t = q_t = 1} \log \left[\frac{p_t(1 - u_t)}{u_t(1 - p_t)} \right] = \sum_{t: x_t = q_t = 1} c_t$$

- $p_t = P(t|\text{Relevant})$ (prob. term occurs in a relevant doc).
 - $u_t = P(t|\text{Non-relevant})$ (prob. term occurs in a non-relevant doc).
 - c_t is the log odds ratio (term weight).
- **Probability Estimates in Practice:**

- u_t (non-relevant) is approximated by global collection statistics: $u_t \approx df_t/N$.
- p_t (relevant) can be set as a constant (e.g., $p_t = 0.5$), or estimated from known relevant documents (if available from RF).

12.4 Okapi BM25 (BestMatch25)

- **Model Type:** A **nonbinary** probabilistic model.
- **Context:** BIM is poor for modern full-text search because it ignores term frequency and document length.
- **Relevance to BM25:** Widely used and robust. Sensitive to term frequency $tf_{t,d}$ and document length L_d .
- **Score Formula (Simplified, without query term weighting):**

$$\text{Score}(d, q) = \sum_{t \in q} \log \left(\frac{N}{df_t} \right) \cdot \frac{tf_{t,d}(k_1 + 1)}{tf_{t,d} + k_1 \left((1 - b) + b \frac{L_d}{L_{ave}} \right)}$$

- L_d/L_{ave} : document length normalized by average length.
- k_1, b : tuning parameters ($k_1 \in [1.2, 2.0]$, $b = 0.75$ common).

12.5 Language Models (LMs) for IR

- **Model Type:** A generative model.
- **Goal:** To determine which document d is most likely to have generated the query q .
- **Ranking:** Documents are ranked by $P(d|q)$. By Bayes' Rule: $P(d|q) \propto P(q|d)P(d)$.
- **Key Probability to Compute:** $P(q|d)$: the probability that a query q would be observed as a random sample from the document model M_d .
- **Unigram Model Assumption:** The generation of a query is an independent process for each term t :

$$P(q|M_d) = \prod_{t \in q} P(t|M_d)^{tf_{t,q}}$$

- **Maximum Likelihood Estimate (MLE):** $P(t|M_d) = tf_{t,d}/|d|$.
- **Problem with MLE:** If $tf_{t,d} = 0$ for any query term t , then $P(q|M_d) = 0$ (the term gets veto power).

12.5.1 Smoothing (Mixture Model)

- **Concept:** Avoids zero probabilities by mixing the document probability with the general collection probability.
- **Formula:** $P(t|d) = \lambda P(t|M_d) + (1 - \lambda)P(t|M_c)$
- $P(t|M_d)$: MLE probability from document d .
- $P(t|M_c)$: MLE probability from the whole collection c (used as a "prior").
- λ : Interpolation parameter (tuning parameter). Setting λ is crucial for performance.
- **Effect:** Has an effect similar to *idf*, where terms rare in the collection but common in the document get higher influence.

13 Crawling and Link Analysis (Lecture 12)

13.1 Crawling

- **Context:** Web search engines must crawl their documents. The sheer magnitude of the web means scaling is a critical problem.
- **Simple Crawler Flow (Initial Idea):** Initialize URL queue $\rightarrow$ loop: fetch page, extract new URLs, add to queue $\rightarrow$ index page.
- **Limitations of Simple Crawler (Problems to Solve):**
 1. **Politeness:** Must respect `robots.txt` protocol (which URLs not to crawl) and space out requests for a single site to avoid overwhelming the server.
 2. **Duplicates:** Must detect and manage both exact and near-duplicates to ensure freshness and efficiency.
 3. **Scale:** Requires a distributed approach (Distributed Crawling Architecture).
 4. **Spider Traps:** Must detect and avoid parts of the web that cause infinite loops or crashes.
- **URL Frontier:** A data structure managing URLs that have been seen but not yet crawled, ensuring politeness constraints (multiple pages from same host are spaced out).
- **URL Normalization:** Must expand all relative URLs (e.g., `aboutsite.html` $\rightarrow$ `http://mit.edu/aboutsite.html`) during parsing for accurate duplicate detection.
- **Distributed Crawling:** Multiple crawl threads (geographically distributed) map target hosts to crawl to specific nodes.

13.2 Link Analysis: Anchor Text

- **Assumption 1:** A hyperlink $d_1 \rightarrow d_2$ is a **quality signal** (author of d_1 deems d_2 high-quality).
- **Assumption 2:** The **anchor text** describes the content of the linked-to page d_2 .
- **Importance of Anchor Text:** Searching on [text of d_2] + [anchor text $\rightarrow d_2$] is often more effective than on d_2 text alone, especially for graphic-heavy pages (e.g., IBM homepage).
- **Anchor Text Indexing:** Anchor text can be indexed and weighted more highly than document text.
- **Google Bombs:** Manipulation of anchor text across many links can lead to undesired/malicious results (e.g., [dangerous cult] linking to a target page).

13.3 Link Analysis: PageRank

- **Concept:** The importance of a page is determined by the importance of the pages that link to it (recursive definition, similar to citation rank).
- **Random Surfer Model:** The score is the "long-term visit rate" of a hypothetical browser doing a random walk on the web.
- **Markov Chain:** The web structure is modeled as a Markov chain with n states (pages) and an $n \times n$ transition probability matrix A .
- **Issues with Simple Random Walk:**
 1. **Dead Ends:** Pages with no outlinks where the walk can get stuck.
 2. **Spider Traps:** Cycles where the walk can get trapped.
- **Solution: Teleporting (Damping Factor α):**
 - With probability α (e.g., 10%), teleport to a random web page (if a dead end, or a random step).
 - With probability $(1 - \alpha)$ (e.g., 90%), follow a random link on the page.
- **Ergodic Markov Chain:** Teleporting ensures the Markov chain is **ergodic** (a path exists from any state to any other), guaranteeing a unique **stable long term visit rate** (the PageRank vector $\vec{a}$).
- **PageRank Calculation:** The stable vector $\vec{a}$ is the **principal eigenvector** of the transition matrix A : $\vec{a} = \vec{a}A$. The rank is computed iteratively: $\vec{x}^{k+1} = \vec{x}^k A$ until $\vec{x}$ stabilizes.

- **Query Processing:** PageRank is a query-independent score $g(d)$ that is combined with query-dependent relevance (e.g., cosine score) for final ranking (e.g., $\text{net-score}(q, d) = g(d) + \cos(q, d)$).
- **Reality:** The original PageRank score is only one component; variants and other features (anchor text, proximity, etc.) are at least as important.

14 Search Advertising and Duplicate Detection (Lecture 13)

14.1 Search Advertising

- **Context:** Search is a key enabler for the web's ecosystem (content creation, financing). Search ads pay for a large part of the web.
- **1st Generation (Goto, 1996):** Pages ranked purely by bid price (**bid**); no separation of ads/docs; revenue maximization.
- **2nd Generation (Google, 2000):** Separates search results (organic) and ads (paid). Ads are ranked based on a combination of bid price and **relevance**.
- **Key Measure of Ad Relevance: Click-Through Rate (CTR)** = clicks per impression.
- **Goal:** Maximizing overall revenue and user acceptance/satisfaction by punishing nonrelevant ads.
- **Google's Second Price Auction:**
 - **Ad Rank:** $\text{bid} \times \text{CTR}$ (trades off money willing to pay vs. ad relevance).
 - **Price Paid (CPC):** The advertiser pays the minimum amount necessary to maintain their position (plus 1 cent).
 - **Formula:** $P_1 = \text{bid}_2 \times \frac{\text{CTR}_2}{\text{CTR}_1}$
- **Keyword Arbitrage:** Buying a keyword for a low price, then redirecting traffic to a page full of ads that pay a much higher price (a form of adversarial IR).
- **Trademark Violations:** Bidding on competitor trademarks. Legal rulings differ globally (e.g., lost in US, won in Europe).

14.2 Duplicate Detection

- **Context:** The web is full of duplicated/near-duplicated content.
- **Exact Duplicates:** Easy to detect using a hash/fingerprint (e.g., MD5).

- **Near-Duplicates:** More common and difficult to eliminate. Annoying to users, as their **marginal relevance is zero**.
- **Syntactic Similarity (Goal):** Measure similarity based on text structure, not true semantic similarity (which is too complex).

14.2.1 Shingling and Jaccard Coefficient

- **Shingle:** A word n -gram (e.g., $n = 3$).
- **Document Representation:** A document is represented as a **set of shingles**.
- **Similarity Measure:** The similarity between two documents d_1, d_2 is the Jaccard coefficient of their shingle sets: $J(d_1, d_2) = \frac{|S_1 \cap S_2|}{|S_1 \cup S_2|}$.
- **Detection:** Documents are near-duplicates if $J(d_1, d_2) > \theta$ (e.g., $\theta = 80\%$).

14.2.2 Min-Hashing (for Fast Jaccard Approximation)

- **Problem:** The set of shingles is very large; direct comparison is too slow.
- **Concept:** Compute a **sketch** (a small vector) for each document, where the similarity of the sketches approximates the true Jaccard similarity.
- **Sketch Element:** A sketch is a vector of n elements (e.g., $n = 200$) derived from n random permutations $(\pi_1, \dots, \pi_n)$ of the shingle space:

$$\text{Sketch}(d) = \langle \min_{s \in d} \pi_1(s), \dots, \min_{s \in d} \pi_n(s) \rangle$$

- **Approximation Property:** The probability that the i^{th} elements of two sketches are equal is approximately the Jaccard similarity: $P(\text{Sketch}(d_1)[i] = \text{Sketch}(d_2)[i]) \approx J(d_1, d_2)$.
- **Jaccard Estimation:** The estimated Jaccard similarity is the fraction of matching components in the two sketches: $\hat{J}(d_1, d_2) = \frac{\# \text{ of matching sketch elements}}{n}$.